

TASK1

Step 1 — Lowercasing

```
text = "Hello!!! My name is FAZIYA. I am learning NLP, and it's AMAZING!!! 😊😊 Running  
text = text.lower()  
print(text)
```

hello!!! my name is faziya. i am learning nlp, and it's amazing!!! 😊😊 running, runs

Step 2 — Remove Punctuation & Special Characters

```
import re  
  
text = re.sub(r'^a-z\s', '', text)  
print(text)
```

hello my name is faziya i am learning nlp and its amazing running runs ran

Step 3 — Tokenization

```
from nltk.tokenize import word_tokenize  
import nltk  
  
nltk.download('punkt')  
  
tokens = word_tokenize(text)  
print(tokens)
```

```
['hello', 'my', 'name', 'is', 'faziya', 'i', 'am', 'learning', 'nlp', 'and', 'its', 'a  
[nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data] Package punkt is already up-to-date!
```

Step 4 — Remove Stopwords

```
from nltk.corpus import stopwords  
  
nltk.download('stopwords')  
  
stop_words = set(stopwords.words('english'))  
  
tokens = [word for word in tokens if word not in stop_words]  
  
print(tokens)
```

```
['hello', 'name', 'faziya', 'learning', 'nlp', 'amazing', 'running', 'runs', 'ran']
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

Step 5 — Lemmatization

```
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()

tokens = [lemmatizer.lemmatize(word, pos='v') for word in tokens]

print(tokens)

['hello', 'name', 'faziya', 'learn', 'nlp', 'amaze', 'run', 'run', 'run']
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

COMPLETE PIPELINE

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

text = "Hello!!! My name is FAZIYA. I am learning NLP, and it's AMAZING!!! 😊😊 Runn

# Step 1: Lowercase
text = text.lower()

# Step 2: Remove punctuation
text = re.sub(r'^a-z\s]', '', text)

# Step 3: Tokenize
tokens = word_tokenize(text)

# Step 4: Remove stopwords
stop_words = set(stopwords.words('english'))
tokens = [word for word in tokens if word not in stop_words]

# Step 5: Lemmatize
lemmatizer = WordNetLemmatizer()
tokens = [lemmatizer.lemmatize(word, pos='v') for word in tokens]

print(tokens)
```

```
['hello', 'name', 'faziya', 'learn', 'nlp', 'amaze', 'run', 'run', 'run']  
[nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data] Package punkt is already up-to-date!  
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data] Package stopwords is already up-to-date!  
[nltk_data] Downloading package wordnet to /root/nltk_data...  
[nltk_data] Package wordnet is already up-to-date!
```

TASK2

Step 1: Store Sentences

```
sentences = [  
    "I saw the man with the telescope.",  
    "Oh wonderful, my internet stopped during the interview.",  
    "He kicked the bucket yesterday.",  
    "BRB, I need to answer a call.",  
    "Chat is my favorite animal."  
]
```

Step 2: Loop Through Each Sentence

```
for sentence in sentences:  
    print("Sentence :", sentence)
```

```
Sentence : I saw the man with the telescope.  
Sentence : Oh wonderful, my internet stopped during the interview.  
Sentence : He kicked the bucket yesterday.  
Sentence : BRB, I need to answer a call.  
Sentence : Chat is my favorite animal.
```

Step 3: Split Sentence into Tokens

```
tokens = sentence.split()  
  
print("Tokens :", tokens)
```

```
Tokens : ['Chat', 'is', 'my', 'favorite', 'animal.']
```

```
for sentence in sentences:  
    tokens = sentence.split()  
  
    if "telescope." in tokens:  
        challenge = "Ambiguity"  
        reason = "Sentence has multiple possible meanings."  
  
    elif "wonderful," in tokens:  
        challenge = "Sarcasm"  
        reason = "Positive word used in a negative situation."
```

```

elif "bucket" in tokens:
    challenge = "Idiom"
    reason = "Phrase meaning differs from literal words."

elif "BRB," in tokens:
    challenge = "Abbreviation/Slang"
    reason = "BRB means Be Right Back."

elif "Chat" in tokens:
    challenge = "Multiple Language Issue"
    reason = "Chat means conversation in English and cat in French."

else:
    challenge = "None"
    reason = "No specific NLP challenge detected."

print("Challenge:", challenge)
print("Reason:", reason)
print("-" * 50)

```

```

Challenge: Ambiguity
Reason: Sentence has multiple possible meanings.
-----
Challenge: Sarcasm
Reason: Positive word used in a negative situation.
-----
Challenge: Idiom
Reason: Phrase meaning differs from literal words.
-----
Challenge: Abbreviation/Slang
Reason: BRB means Be Right Back.
-----
Challenge: Multiple Language Issue
Reason: Chat means conversation in English and cat in French.
-----

```

TASK3

Part A: Preprocess

```

docs = [
    "I love pani puri",
    "I love momos",
    "pani puri is my favorite junkfood"
]

```

Step 1: Lowercase

```

docs = [doc.lower() for doc in docs]
print(docs)

```

```
['i love pani puri', 'i love momos', 'pani puri is my favorite junkfood']
```

Step 2: Tokenize

```
tokens = [doc.split() for doc in docs]
print(tokens)
```

```
[['i', 'love', 'pani', 'puri'], ['i', 'love', 'momos'], ['pani', 'puri', 'is', 'my', '']
```

Step 3: Remove Stopwords

```
stopwords = ['i', 'is', 'my']

clean_docs = []

for doc in tokens:
    words = []
    for word in doc:
        if word not in stopwords:
            words.append(word)
    clean_docs.append(words)

print(clean_docs)
```

```
[['love', 'pani', 'puri'], ['love', 'momos'], ['pani', 'puri', 'favorite', 'junkfood']
```

Part B: Create Vocabulary

```
vocab = []

for doc in clean_docs:
    for word in doc:
        if word not in vocab:
            vocab.append(word)

print(vocab)
```

```
['love', 'pani', 'puri', 'momos', 'favorite', 'junkfood']
```

Part C: Build Bag of Words

```
bow = []

for doc in clean_docs:
    row = []

    for word in vocab:
        row.append(doc.count(word))

    bow.append(row)
```

```
print(bow)
```

```
[[1, 1, 1, 0, 0, 0], [1, 0, 0, 1, 0, 0], [0, 1, 1, 0, 1, 1]]
```

Double-click (or enter) to edit

```
import math

docs = [
    "I love pani puri",
    "I love momos",
    "pani puri is my favorite junkfood"
]

docs = [doc.lower() for doc in docs]

print("After Lowercasing:")
print(docs)

tokenized_docs = []

for doc in docs:
    tokenized_docs.append(doc.split())

print("\nAfter Tokenization:")
print(tokenized_docs)

stopwords = ["i", "is", "my"]

clean_docs = []

for doc in tokenized_docs:
    words = []

    for word in doc:
        if word not in stopwords:
            words.append(word)

    clean_docs.append(words)

print("\nAfter Stopword Removal:")
print(clean_docs)

vocab = []
```

```
for doc in clean_docs:
    for word in doc:
        if word not in vocab:
            vocab.append(word)

print("\nVocabulary:")
print(vocab)

bow_matrix = []

for doc in clean_docs:

    row = []

    for word in vocab:
        row.append(doc.count(word))

    bow_matrix.append(row)

print("\nBag of Words Matrix:")

for i, row in enumerate(bow_matrix):
    print(f"Doc{i+1}:", row)

tf_matrix = []

for doc in clean_docs:

    doc_tf = []

    total_words = len(doc)

    for word in vocab:
        tf = doc.count(word) / total_words
        doc_tf.append(round(tf, 3))

    tf_matrix.append(doc_tf)

print("\nTF Matrix:")

for i, row in enumerate(tf_matrix):
    print(f"Doc{i+1}:", row)

N = len(clean_docs)

idf = {}

for word in vocab:
```

```
df = 0

for doc in clean_docs:
    if word in doc:
        df += 1

idf[word] = round(math.log(N / df), 3)

print("\nIDF Values:")

for word, value in idf.items():
    print(word, ":", value)

tfidf_matrix = []

for doc in clean_docs:

    row = []

    total_words = len(doc)

    for word in vocab:

        tf = doc.count(word) / total_words

        tfidf = tf * idf[word]

        row.append(round(tfidf, 3))

    tfidf_matrix.append(row)

print("\nTF-IDF Matrix:")

for i, row in enumerate(tfidf_matrix):
    print(f"Doc{i+1}:", row)

highest_word = ""
highest_score = 0

for word in vocab:

    for row in tfidf_matrix:

        index = vocab.index(word)

        if row[index] > highest_score:
            highest_score = row[index]
            highest_word = word

print("\nHighest TF-IDF Word:")
print(highest_word, "=", highest_score)
```



```

rare_words = []

for word in vocab:

    count = 0

    for doc in clean_docs:
        if word in doc:
            count += 1

    if count == 1:
        rare_words.append(word)

print("\nWords Appearing In Only One Document:")
print(rare_words)

```

After Lowercasing:

```
['i love pani puri', 'i love momos', 'pani puri is my favorite junkfood']
```

After Tokenization:

```
[['i', 'love', 'pani', 'puri'], ['i', 'love', 'momos'], ['pani', 'puri', 'is', 'my', 'junkfood']]
```

After Stopword Removal:

```
[['love', 'pani', 'puri'], ['love', 'momos'], ['pani', 'puri', 'favorite', 'junkfood']]
```

Vocabulary:

```
['love', 'pani', 'puri', 'momos', 'favorite', 'junkfood']
```

Bag of Words Matrix:

```
Doc1: [1, 1, 1, 0, 0, 0]
```

```
Doc2: [1, 0, 0, 1, 0, 0]
```

```
Doc3: [0, 1, 1, 0, 1, 1]
```

TF Matrix:

```
Doc1: [0.333, 0.333, 0.333, 0.0, 0.0, 0.0]
```

```
Doc2: [0.5, 0.0, 0.0, 0.5, 0.0, 0.0]
```

```
Doc3: [0.0, 0.25, 0.25, 0.0, 0.25, 0.25]
```

IDF Values:

```
love : 0.405
```

```
pani : 0.405
```

```
puri : 0.405
```

```
momos : 1.099
```

```
favorite : 1.099
```

```
junkfood : 1.099
```

TF-IDF Matrix:

```
Doc1: [0.135, 0.135, 0.135, 0.0, 0.0, 0.0]
```

```
Doc2: [0.203, 0.0, 0.0, 0.549, 0.0, 0.0]
```

```
Doc3: [0.0, 0.101, 0.101, 0.0, 0.275, 0.275]
```

Highest TF-IDF Word:

```
momos = 0.549
```

Words Appearing In Only One Document:

```
['momos', 'favorite', 'junkfood']
```

